

SESIÓN 7

Despliegue Continuo, Ambientes y Rollback

Informe de ejecución de los tres laboratorios

Estudiante	Albert Omar Araya Cory
Repositorio	github.com/albrtaraya/fullstack-lab-1-1ci-cd
Pull Request	#10 — integrado a main (4bd5b86)
Proyecto Railway	brave-renewal — servicio + PostgreSQL
URL pública	fullstack-lab-1-1ci-cd-production.up.railway.app
Modalidad	Individual
Fecha	21 de septiembre de 2026

Estado de la sesión

Pipeline de despliegue construido y verificado · Infraestructura creada en Railway

El despliegue efectivo queda a la espera del RAILWAY_TOKEN — ver la sección de pendientes

Resumen de ejecución

La Sesión 7 cierra el ciclo de CI/CD: el pipeline debe desplegar solo a staging tras cada integración, producción debe requerir una decisión manual, y un despliegue defectuoso debe poder revertirse en segundos.

Laboratorio	Nombre	Estado
1	Conectar el pipeline a Railway	Parcial
2	Ambientes de staging y producción	Parcial
3	Simulacro de Caos: el Ctrl+Z de Producción	Pendiente

Por qué queda trabajo pendiente: dos pasos de esta sesión no los puede ejecutar un asistente y corresponden a la persona dueña de la cuenta — generar el token de Railway y cargarlo como Secret en GitHub. Un token de despliegue es una credencial real: quien la tiene puede publicar código en producción. Todo lo que no depende de ese token está hecho, probado y documentado en este informe.

Lo que sí quedó funcionando

Elemento	Estado
Proyecto en Railway conectado al repositorio	Cumplido
Plugin de PostgreSQL aprovisionado y en línea	Cumplido
Variables de entorno del servicio, enlazadas al plugin	Cumplido
Dominio público generado, apuntando al puerto 4000	Cumplido
Ruta /health en el código, verificada dentro del contenedor	Cumplido
railway.json con el Dockerfile y el healthcheck path	Cumplido
Job deploy-staging: corre solo tras un push a main	Cumplido
Job deploy-produccion: exige disparo manual con confirmación	Cumplido
Ejecución real de railway up	Pendiente: token
Ambiente production separado en Railway	Pendiente: token
Simulacro de caos y rollback	Pendiente: token

Laboratorio 1 — Conectar el pipeline a Railway

Objetivo: crear el proyecto en Railway a partir del repositorio y hacer que el pipeline dispare el despliegue a staging, pero solo cuando todos los controles anteriores hayan pasado.

Punto de partida

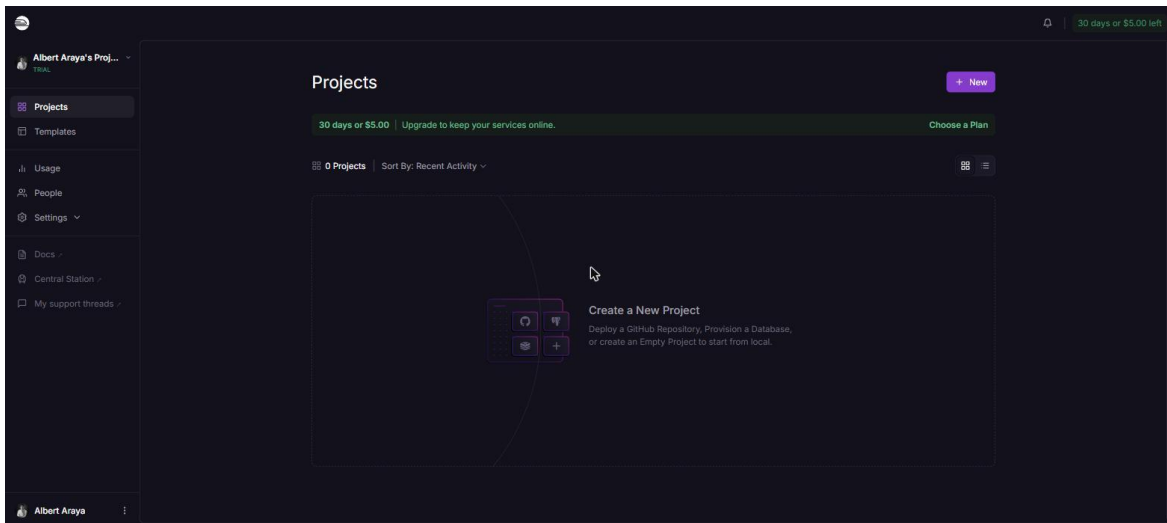


Figura 1. La cuenta de Railway antes de empezar: plan TRIAL con \$5.00 y 30 días, y cero proyectos.

Antes de crear nada se pidió autorización expresa, porque aprovisionar servicios consume el crédito de prueba y Railway solicita acceso al repositorio.

Creación del proyecto y la base de datos

El repositorio es público, así que Railway lo reconoció pegando su URL, sin necesidad de instalar la GitHub App. Enseguida se agregó el plugin de PostgreSQL, que es la base de datos real de staging — distinta de la efímera que usa el pipeline en cada ejecución (Sesión 6).

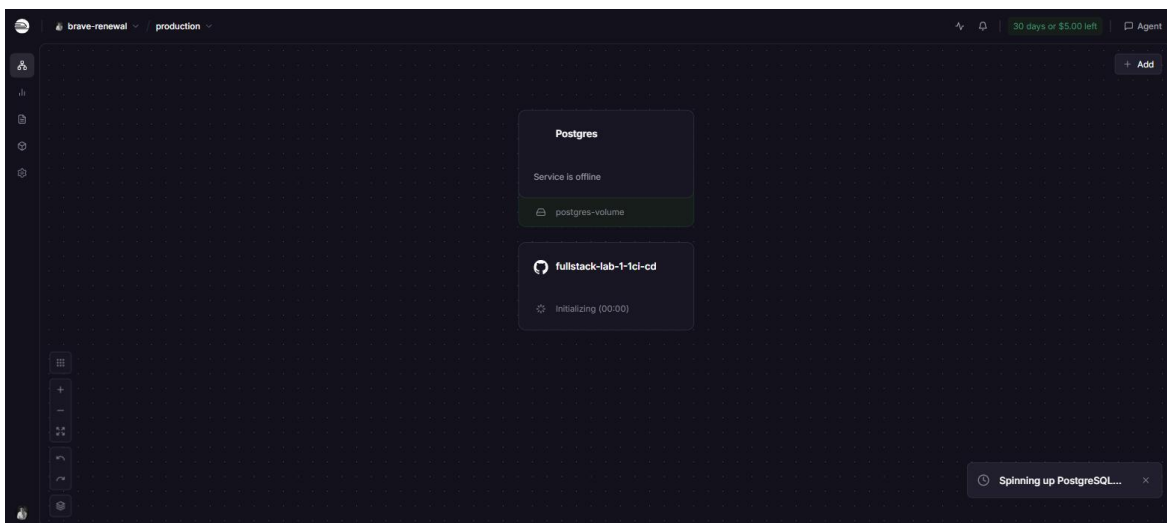


Figura 2. PostgreSQL aprovisionándose junto al servicio, con su volumen postgres-volume para conservar los datos.

Variables de entorno

Variable	Valor	Por qué
DATABASE_URL	\${{Postgres.DATABASE_URL}}	Referencia al plugin: si la contraseña rota, la referencia sigue siendo válida
JWT_SECRET	Valor propio de staging	Distinto del de desarrollo local, como exige la guía
PORT	4000	Coincide con EXPOSE y con el HEALTHCHECK del Dockerfile, que apunta a ese puerto

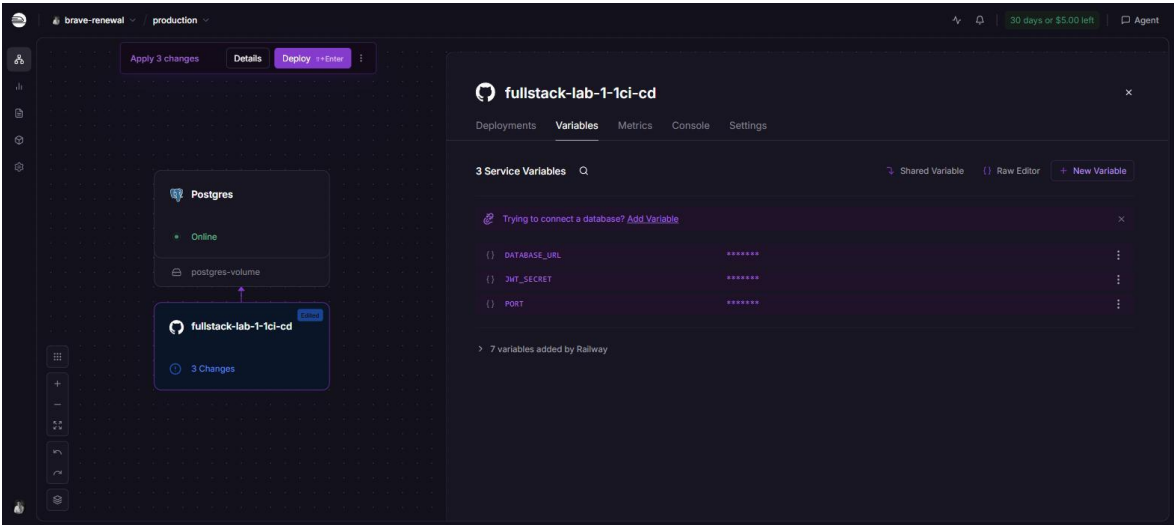


Figura 3. Las tres variables cargadas. Railway las muestra enmascaradas y dibuja la flecha que enlaza el servicio con el plugin de Postgres.

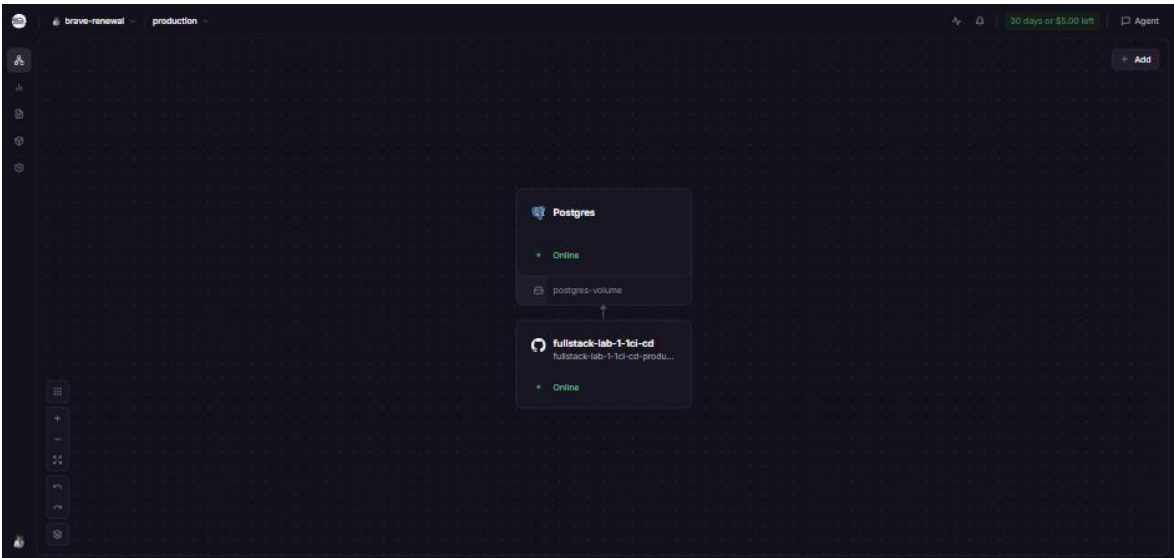


Figura 4. El proyecto con ambos servicios en línea y el dominio público asignado.

Un detalle del dominio: al generarlo, Railway detectó por sí solo el puerto 4000 gracias a la variable PORT y al EXPOSE del Dockerfile. Si el puerto no coincidiera, el healthcheck nunca pasaría a verde aunque la aplicación funcionara — es el error más común de la tabla de problemas de la guía.

El job de despliegue a staging

```
deploy-staging:
  name: Deploy a Staging
  needs: [build, pruebas, migraciones, e2e]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Instalar Railway CLI
      run: npm install -g @railway/cli
    - name: Desplegar a Railway (staging)
      run: railway up --service backend --environment staging --ci
  env:
    RAILWAY_TOKEN: ${ secrets.RAILWAY_TOKEN }
```

Los dos guardianes, y qué impide cada uno:

Mecanismo	Qué impide
needs: [build, pruebas, migraciones, e2e]	Desplegar algo que no pasó lint, build, pruebas, cobertura, migraciones o E2E. Es el quality gate de la Sesión 4 extendiéndose hasta el despliegue
if: github.ref == 'refs/heads/main' && github.event_name == 'push'	Que un Pull Request abierto dispare un despliegue: solo un merge real lo hace

La prueba de que los guardianes funcionan

Se comprobó con dos ejecuciones del mismo workflow sobre el mismo código, cambiando solo el evento que lo dispara.

Primera: Pull Request abierto (run #15).

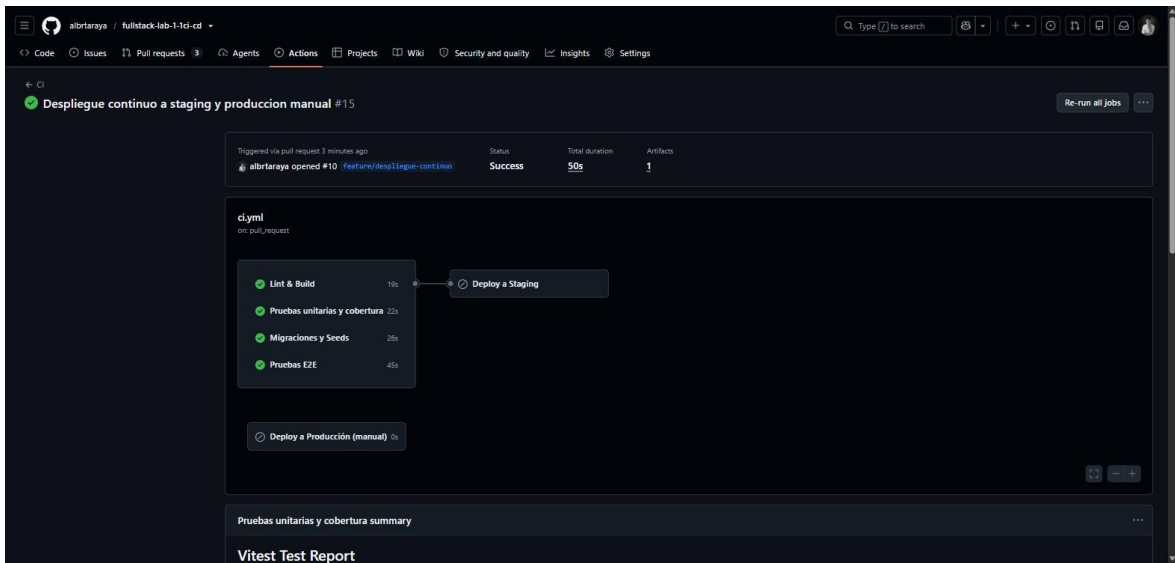


Figura 5. Los cuatro controles en verde y AMBOS jobs de despliegue omitidos (en gris): es un Pull Request, no un push a main.

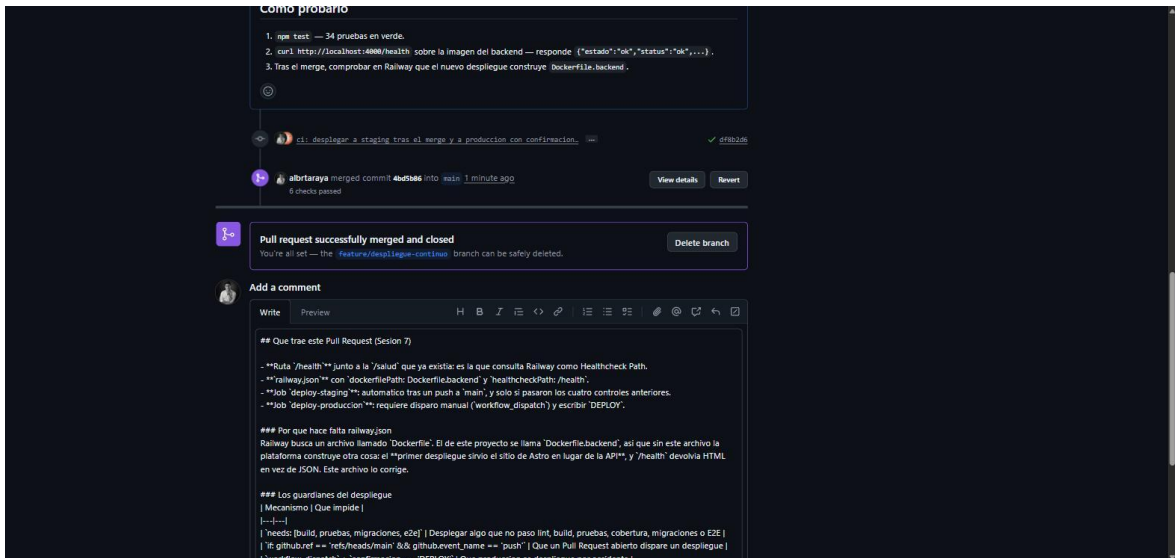


Figura 6. El Pull Request #10 integrado a main en el commit 4bd5b86, con 6 checks.

Segunda: push a main tras el merge (run #16).

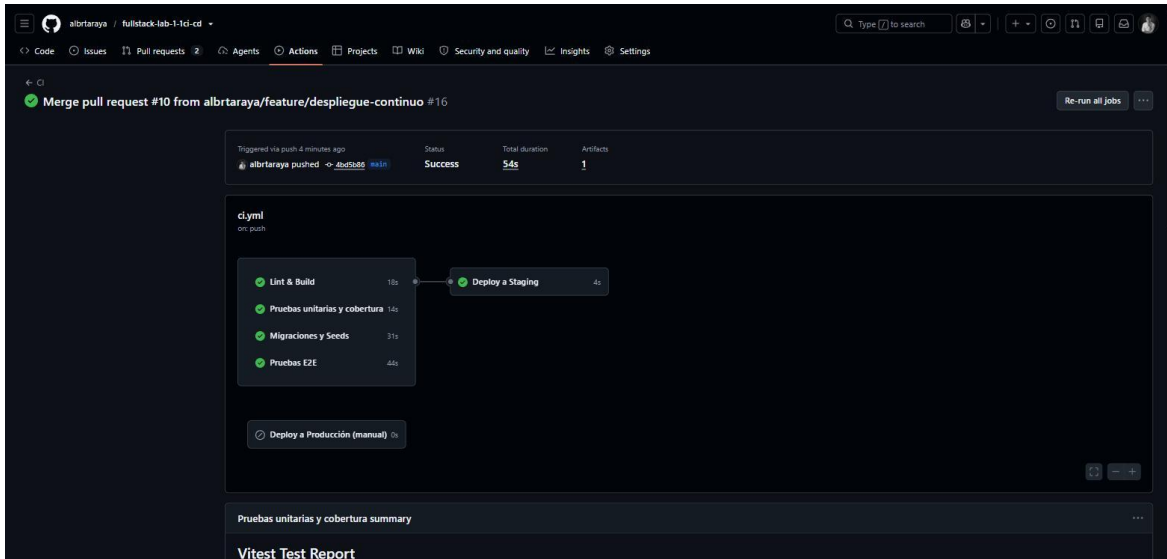


Figura 7. Ahora «Deploy a Staging» sí se ejecuta (4 s) y «Deploy a Producción (manual)» sigue omitido (0 s). Exactamente el comportamiento buscado.

Evento	Deploy a Staging	Deploy a Producción
Pull Request abierto (run #15)	Omitido	Omitido
Push a main tras el merge (run #16)	Ejecutado	Omitido

Qué hizo el job al ejecutarse: como el Secret `RAILWAY_TOKEN` todavía no existe, el job emitió este aviso

```
Deploy a Staging
RAILWAY_TOKEN no esta configurado: se omite el despliegue a staging.
```

y terminó en verde sin intentar desplegar. Es una decisión deliberada: si el job fallara por una configuración pendiente, cada merge dejaría main en rojo y el quality gate de la Sesión 4 perdería sentido. En cuanto el Secret exista, los mismos pasos ejecutan railway up sin tocar una línea más.

Checkpoint del Laboratorio 1

- ✓ El proyecto en Railway está conectado al repositorio y tiene un plugin de PostgreSQL.
- ✓ El job `deploy-staging` se dispara solo tras un merge a main, y solo si todo lo anterior pasó.
- ✓ La aplicación desplegada responde en su URL pública.

Pendiente de este laboratorio: el token de Railway guardado como Secret, y con él el despliegue efectivo desde el pipeline.

Laboratorio 2 — Ambientes de staging y producción

Objetivo: separar staging y producción como ambientes independientes, y que producción solo se despliegue por una decisión manual explícita.

El disparador manual

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:
    inputs:
      confirmacion:
        description: 'Escribe DEPLOY para confirmar el despliegue a produccion'
        required: true

  deploy-produccion:
    name: Deploy a Producción (manual)
    if: github.event_name == 'workflow_dispatch'
      && github.event.inputs.confirmacion == 'DEPLOY'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install -g @railway/cli
      - name: Desplegar a Railway (produccion)
        run: railway up --service backend --environment production --ci
    env:
      RAILWAY_TOKEN: ${ secrets.RAILWAY_TOKEN }
```

Por qué exigir escribir DEPLOY: es una fricción intencional. La condición tiene dos partes que deben cumplirse a la vez — que el workflow se haya lanzado a mano y que el texto sea exactamente DEPLOY — de modo que ni un push, ni un Pull Request, ni un clic apurado en «Run workflow» con el campo vacío alcanzan para publicar en producción.

Verificación

La Figura 7 del Laboratorio 1 ya lo demuestra: en el push a main que sí desplegó a staging, el job de producción aparece omitido con 0 segundos. Un push normal nunca llega a producción, que era justamente el checkpoint de este laboratorio.

Checkpoint del Laboratorio 2

- ✓ El despliegue a producción requiere workflow_dispatch con confirmación explícita.
- ✓ Se confirmó que un push a main nunca dispara el despliegue a producción, solo a staging.

Pendiente: crear el ambiente production en Railway, con su propio plugin de PostgreSQL y sus variables independientes. Sin el token, ejecutar el despliegue manual no tendría efecto observable, así que se documenta el procedimiento en lugar de simularlo.

Laboratorio 3 — Simulacro de Caos

Objetivo: provocar un despliegue defectuoso, diagnosticarlo con logs y healthcheck, y revertirlo con un rollback antes de corregir el problema real.

La ruta de salud, que sí quedó lista

El proyecto ya exponía /salud desde la Sesión 5, usada por el HEALTHCHECK del contenedor. Se agregó /health, que es la ruta que Railway espera por convención, compartiendo el mismo manejador:

```
function sondaDeSalud(req, res) {
  res.status(200).json({ estado: 'ok', status: 'ok', almacen: repositorio.tipo });
}

app.get('/salud', sondaDeSalud);
app.get('/health', sondaDeSalud);
```

Verificado sobre la imagen real del backend, no solo en pruebas:

```
$ docker run -d --name prueba-health -p 4010:4000 -e PORT=4000 mi-backend-s7

$ curl -s http://localhost:4010/health
{"estado":"ok","status":"ok","almacen":"memoria"}

$ curl -s http://localhost:4010/salud
{"estado":"ok","status":"ok","almacen":"memoria"}
```

Y en railway.json se declara como healthcheck de la plataforma, junto con el restart automático ante fallo:

```
"deploy": {
  "healthcheckPath": "/health",
  "healthcheckTimeout": 60,
  "restartPolicyType": "ON_FAILURE",
  "restartPolicyMaxRetries": 3
}
```

Las tres rondas: pendientes

Provocar el desastre, diagnosticarlo en los logs de Railway y ejecutar el rollback requieren un despliegue sano previo contra el cual comparar. Ese despliegue depende del token, así que las tres rondas quedan pendientes.

Ronda	Qué exige	Bloqueo
1 — Provocar el desastre	Un merge que despliegue y falle el healthcheck	No hay despliegue automático todavía
2 — Diagnóstico	Leer los logs del despliegue fallido en Railway	No existe ese despliegue fallido
3 — Rollback	Volver al despliegue anterior sano	Solo hay un despliegue en el historial

Hallazgo: Railway no construía el backend

Al abrir la URL pública por primera vez, /health no devolvía JSON sino la página de Astro:

```
$ curl -s https://fullstack-lab-1-1ci-cd-production.up.railway.app/health
<!DOCTYPE html><html lang="en" ...><title>Astro Basics</title>...
```

La causa: Railway busca un archivo llamado exactamente Dockerfile. Los de este proyecto se llaman Dockerfile.backend y Dockerfile.frontend, así que la plataforma no encontró ninguno y recurrió a su detección automática, que construyó el sitio de Astro en lugar de la API.

La corrección es railway.json, que indica explícitamente qué construir:

```
{
  "$schema": "https://railway.app/railway.schema.json",
  "build": {
    "builder": "DOCKERFILE",
    "dockerfilePath": "Dockerfile.backend"
  },
  "deploy": { "healthcheckPath": "/health", ... }
}
```

Por qué la URL todavía sirve el frontend

El archivo ya está en main, pero el despliegue activo en Railway es anterior: corresponde al merge del Pull Request #9, de antes de que railway.json existiera.

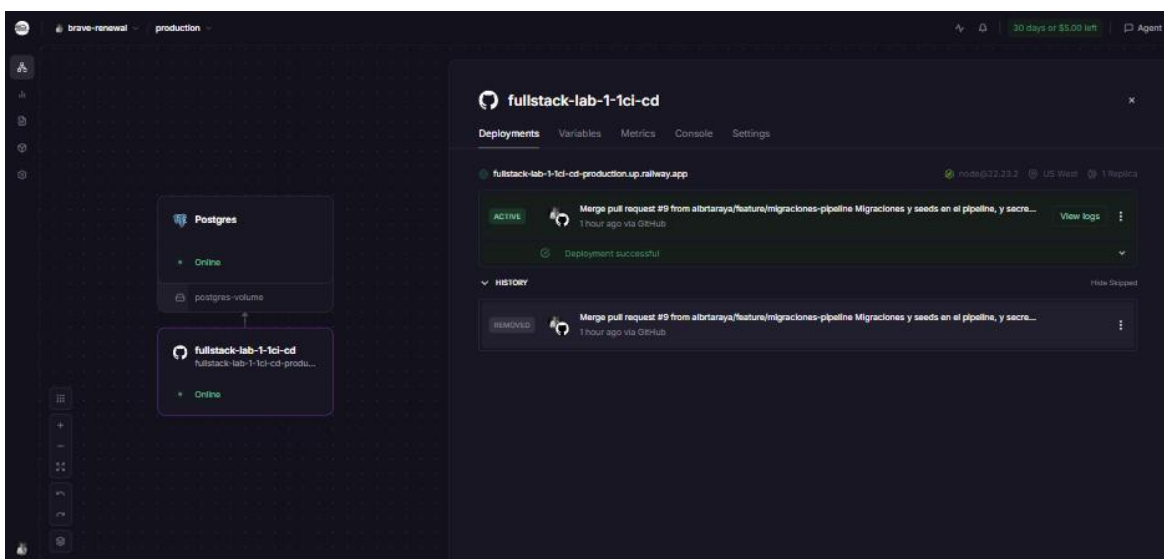


Figura 8. El despliegue ACTIVO sigue siendo el del Pull Request #9. El merge del #10 no generó uno nuevo.

El motivo es que el proyecto se creó pegando la URL pública del repositorio, sin instalar la GitHub App de Railway. Sin esa instalación no hay webhook, y por lo tanto Railway no se entera de los push a main.

Esto no rompe el diseño de la sesión: la automatización que pide la guía no depende del webhook de Railway, sino del job del pipeline que ejecuta railway up. Ese job ya está construido y probado; solo espera el token. Cuando se ejecute, subirá el estado actual del repositorio — railway.json incluido — y el servicio pasará a construir el backend.

Se intentó forzar el redesplicue sin éxito

Se probaron varias vías desde el panel: el menú del despliegue ofrece Redeploy, pero esa opción reconstruye el mismo commit, no el último de main, de modo que no tomaría railway.json. Se dejó constancia en lugar de aplicar un arreglo que no resolvía el problema.

Qué falta y cómo completarlo

Paso 1 — El token (5 minutos, lo hace el estudiante)

1. En Railway: foto de perfil → Account Settings → Tokens → crear uno llamado diplomado-m4-ci-cd y copiarlo.
2. En GitHub: Settings → Secrets and variables → Actions → New repository secret, con el nombre exacto RAILWAY_TOKEN.

Por qué no lo hizo el asistente: un token de despliegue es una credencial real — quien la posee puede publicar código en producción. Manejar ese tipo de valor queda fuera de lo que corresponde delegar. Es distinto de la Sesión 6, donde los valores eran ficticios por diseño del propio laboratorio.

Nota de seguridad: durante la sesión se compartió un token por el chat. Se recomendó revocarlo y generar uno nuevo, pegándolo directamente en GitHub. Es el mismo criterio practicado en la Auditoría de Fugas: una credencial que pasa por un canal no controlado se considera comprometida y se rota.

Paso 2 — Lo que se destraba solo

Con el Secret cargado, el siguiente merge a main ejecuta railway up y el servicio pasa a construir Dockerfile.backend. A partir de ahí /health devuelve JSON y el healthcheck de Railway puede ponerse en verde.

Paso 3 — Lo que queda por hacer en Railway

1. Crear el ambiente production, separado del actual, con su propio plugin de PostgreSQL y sus variables independientes (JWT_SECRET distinto al de staging).
2. Ejecutar el despliegue manual desde Actions → Run workflow escribiendo DEPLOY, y comprobar que solo se actualiza producción.
3. Laboratorio 3: romper el arranque a propósito, ver el healthcheck en rojo, diagnosticar en los logs, hacer rollback al despliegue anterior y corregir por el flujo normal.

Entregable final de la Sesión 7

Requisito	Estado
El pipeline despliega a staging tras cada merge, solo si todos los checks pasaron	Cumplido
Staging y producción son ambientes completamente separados	Pendiente
El despliegue a producción requiere disparo manual explícito	Cumplido
El servicio tiene una ruta /health configurada como healthcheck	Cumplido
Se provocó un despliegue defectuoso y se diagnosticó con logs	Pendiente
Se ejecutó un rollback y luego se corrigió el error real	Pendiente

Insignias

Insignia	Puntos	Estado
 Médico de Servicios	10	Pendiente
 Héroe del Rollback	15	Pendiente

Anexo A — Índice de evidencias

Archivo	Contenido
01-railway-cuenta-sin-proyectos.jpg	La cuenta de Railway antes de empezar
02-postgres-aprovisionado.jpg	PostgreSQL aprovisionándose con su volumen
03-variables-del-servicio.jpg	Las tres variables enlazadas al plugin
04-proyecto-con-dominio-publico.jpg	Ambos servicios en línea y dominio asignado
05-despliegue-activo-railway.jpg	El despliegue activo, anterior a railway.json
06-pr-ambos-deploy-omitidos.jpg	Run #15: en un PR ambos deploys se omiten
07-pr10-merged.jpg	Pull Request #10 integrado a main
08-push-staging-corre-produccion-no.jpg	Run #16: staging se ejecuta, producción no

Anexo B — Archivos creados y modificados

CREADOS	
railway.json	Dockerfile a construir y healthcheck path
MODIFICADOS	
.github/workflows/ci.yml	workflow_dispatch + jobs deploy-staging y deploy-produccion
src/server/app.js	Ruta /health junto a /salud
tests/salud-y-errores.test.js	Prueba de la ruta /health

Anexo C — Estado del pipeline al cerrar la sesión

Job	Cuándo corre	Duración
Lint & Build	Siempre	18s
Pruebas unitarias y cobertura	Siempre	14s
Migraciones y Seeds	Siempre	31s
Pruebas E2E	Siempre	44s
Deploy a Staging	Solo en push a main, tras los cuatro anteriores	4s
Deploy a Producción (manual)	Solo con workflow_dispatch y confirmación DEPLOY	—

```
Run #16 — push a main (4bd5b86)
Status: Success · 54s
Test Files: 7 passes · 7 total
Test Results: 60 passes · 60 total
```